

Assert, Require, or Revert Violation

The Assert, Require, or Revert Violation vulnerability occurs when a smart contract evaluates a condition that leads to an assert, require, or revert instruction being triggered unexpectedly, without proper control, or under conditions that can be influenced by an adversary. A failing assert, require, or revert can be triggered by an attacker, indicate faulty logic, or lead to the unexpected propagation of a revert that breaks invariants or functionality.

An attacker can intentionally trigger failing assertions or conditions, causing the contract to revert critical operations, waste gas, lock tokens or funds, or block certain addresses—or even all users—from interacting with the contract.

Specification primitives Our defect specifications rely on standard Solidity datatypes— `Contract`, `Method`, `Address`, `Env(ironment)`, and `State`. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

Table 1. Primitives for Assert, Require, or Revert Violation Specification

Symbol	Semantics
<i>Static Analysis (Slither)</i>	
<code>methods(c)</code>	The set of public/external methods defined in contract <i>c</i> .
<i>Runtime Verification (LLM-generated based on specific template)</i>	
<code>FailExec(<i>m</i>, σ, σ')</code>	Holds if invoking <i>m</i> in state σ its execution fails in state σ' .
<code>IsPanic(σ)</code>	Holds if a state transition resulted in a “Panic” revert (Solidity 0.8.x), which corresponds to internal logic errors (e.g., assert failure or array out-of-bounds) rather than user-side input validation (require).
<code>IsGriefable(<i>m</i>)</code>	Holds if the method, when forced to fail for all users, result in a permanent Denial-of-Service or locked funds.

Domain Conceptual Model. The Assert/Require/Revert Violation vulnerability occurs when a smart contract evaluates a condition that leads to an assert, require, or revert instruction being triggered unexpectedly, without proper control, or under conditions that can be influenced by an adversary. A failing assert, require, or revert can be triggered by an attacker, indicate a faulty logic, or lead to unexpected propagation of a revert that breaks invariants or functionality.

Formal Specification.

$$\begin{aligned} \text{LogicFault}(c) \equiv & \exists m \in \text{methods}(c). \exists x \in \text{Dom}(m). \exists \sigma, \sigma_{err} \in \text{State}. \\ & \text{FailExec}(m(x), \sigma, \sigma_{err}) \wedge (\text{IsPanic}(\sigma_{err}) \vee \text{IsGriefable}(m) \vee \\ & (\forall A \in \text{Address}. \text{FailExec}(m(x), \sigma, \sigma_{err}))) \end{aligned}$$

This captures Griefing attacks, where an attacker can force a contract into a perpetual failure state for all users, effectively locking funds without necessarily violating a functional spec.

Specific test template design.

Testing Goal Identify valid input sequences that transition the contract into a Panic state or a permanent Denial-of-Service (DoS) condition.

Setup The LLM initializes the contract state and identifies “essential” methods that must remain accessible for the system to function correctly, such as `withdraw()` or critical administrative functions.

Action The test uses fuzzing or symbolic parameters to invoke the target method m in state σ . For reaching specific faults, the LLM may establish a specialized configuration through additional setup calls prior to the main transaction.

Check The harness monitors for Solidity Panic codes (e.g., 0x01 for failed assertions) and unexpected reverts triggered by valid environment states.

Assertion Verification is achieved using the `vm.expectRevert()` cheatcode, which asserts that the call to method m fails under the identified conditions. If a sequence of transactions makes an essential method permanently unreachable or causes it to revert unexpectedly, the framework flags the behavior as a valid vulnerability.

Template Derivation and LLM Instantiation The conceptual test design is materialized into a standardized Foundry test template. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments to guide the LLM through the instantiation phase. The process unfolds as follows:

Target Initialization (Setup): The LLM is instructed to import the target smart contract artifact, initialize the contract under test, and import `stdError` to catch specific Panic codes. It identifies “essential” methods that must remain accessible to prevent DoS conditions.

Input Parameterization (Fuzzing): To support fuzzing or symbolic execution, the LLM analyzes the target method and adds any required arguments to the test function signature. It uses `vm.assume` to constrain these inputs to reachable logical paths, deliberately avoiding assumptions that would make the specific assertion failure impossible.

State Configuration (Setup): The LLM establishes specialized configurations prior to the main transaction. This includes funding the test contract or victim if interacting with payable functions, and forcing the contract into a “contradictory” state using public setters (Strategy A) or storage manipulation like `stdstore` (Strategy B).

Standardized Validation (Check & Assertion): The test framework monitors for unexpected failures. Before invoking the target method, the LLM places `vm.expectRevert(stdError.assertionError)` to assert that the subsequent call will fail under the identified conditions.

Exploit Simulation (Action): The LLM invokes the target method with the fuzzed or symbolic parameters. If the call reverts with the expected Panic code or causes an essential method to become unreachable, it proves the existence of an invalid state or contradiction, successfully flagging the vulnerability.

```

1  // SPDX-License-Identifier: MIT
2  pragma solidity 0.8.29;
3
4  // [LLM_INSTRUCTION]: Import stdError to catch Assertion Violation (Panic 0
   x01).
5  import {Test, stdError} from "../lib/forge-std/src/Test.sol";
6  // [LLM_INSTRUCTION]: If you need to manipulate private state directly,
   import StdStorage:
7  // import {stdStorage, StdStorage} from "../lib/forge-std/src/StdStorage.sol
   ";
8
9  // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
10 // solidity files are in "../src/". The name of the file is the same as the
   name of the contract.
11
12 // -----
13 // [Testing Goal] Identify valid input sequences that transition the contract
14 // into a Panic state or a permanent Denial-of-Service (DoS) condition.
15 // -----
16
17 // [LLM_INSTRUCTION]: Name the contract 'TestAssertFailure[ContractName]'
18 contract TestAssertFailureTemplate is Test {
19     // [LLM_INSTRUCTION]: Use StdStorage if needed for complex state setup:
20     // using stdStorage for StdStorage;
21
22     // -----
23     // [Setup] Declare the contract under test. Identify "essential" methods
24     // that must remain accessible (e.g., withdraw(), admin functions).
25     // -----
26
27     // [LLM_INSTRUCTION]: Declare the contract under test variable
28     // ConditionAlwaysFalse public _contractUnderTest;
29
30     // ----- [/Setup]
31
32     function setUp() public {
33         // -----
34         // [Setup] Initialize the contract state.
35         // -----
36
37         // [LLM_INSTRUCTION]: Initialize the contract under test.

```

```

38         // 1. If constructor parameters are needed, use concrete valid values
39         .
40         // 2. If payable, use vm.deal(address(this), amount) before
           deployment.
41         // _contractUnderTest = new ConditionAlwaysFalse();
42         // ----- [/Setup]
43     }
44
45     // [LLM_INSTRUCTION]: Analyze the method being tested.
46     // 1. If it accepts arguments, ADD them to this function signature to
           enable Fuzzing/Symbolic execution.
47     // 2. If it takes no arguments, keep the signature empty.
48     // Example: function test_highlightAssertionFailure(uint256 fuzzArg)
           public {
49     function test_highlightAssertionFailure() public {
50
51         // -----
52         // [Setup] Constrain fuzz/symbolic inputs and configure pre-
           conditions.
53         // The LLM may establish a specialized configuration through
           additional
54         // setup calls prior to the main transaction.
55         // -----
56
57         // [LLM_INSTRUCTION]: Use 'vm.assume' to constrain inputs to
           reachable logical paths.
58         // WARNING: Avoid assumptions that make the specific assertion
           failure impossible.
59         // Example: vm.assume(fuzzArg > 10);
60
61         // [LLM_INSTRUCTION]: FUNDING (If Applicable)
62         // Even for logic tests, funding ensures calls don't fail due to low-
           level balance checks.
63         // 1. Fund the Victim: vm.deal(address(_contractUnderTest), 100 ether
           );
64         // 2. CRITICAL: Fund the Test Contract (address(this)) if it
           interacts with payable functions.
65         // vm.deal(address(this), 100 ether);
66
67         // [LLM_INSTRUCTION]: STATE VARIABLES
68         // Does the assertion failure require specific state (e.g. an
           inconsistent update)?
69         // STRATEGY A: Public Methods (Preferred)
70         // Call public setters.
71         // Example: _contractUnderTest.setState(fuzzArg);
72
73         // STRATEGY B: Storage Manipulation (For Private/Hard-to-reach State)
74         // Use 'vm.store' or 'stdstore' to force the contract into a "
           contradictory" state.
75         // Example: stdstore.target(address(_contractUnderTest)).sig("myVar()
           ").checked_write(fuzzArg);
76
77         // ----- [/Setup]
78
79         // -----
80         // [Check] Monitor for Solidity Panic codes (e.g., 0x01 for failed
           assertions)
81         // and unexpected reverts triggered by valid environment states.
82         // -----
83

```

```

84 // -----
85 // [Assertion] Use vm.expectRevert() to assert that the call to the
86 // target method fails under the identified conditions. If a sequence
87 // of
88 // transactions makes an essential method permanently unreachable or
89 // causes it to revert unexpectedly, the behavior is flagged as a
90 // valid vulnerability.
91 // -----
92 // This confirms that the contract reverts with Panic(0x01) (Assert
93 // Failed).
94 // This proves the existence of a contradiction or invalid state.
95 vm.expectRevert(stdError.assertionError);
96 // ----- [/Check]
97 // -----
98 // [Action] Invoke the target method with fuzzing/symbolic parameters
99 // to trigger the fault.
100 // -----
101 // [LLM_INSTRUCTION]: Call the function.
102 // If you added parameters to the test signature, pass them here.
103 // _contractUnderTest.notGonnaExecute(fuzzArg);
104 // ----- [/Action]
105 // ----- [/Assertion]
106 }
107 }
108 }

```

Listing 1.1. Assert, Require, or Revert Violation Multi-Shot test template